

CS6160: Cryptology

Programming Assignment 2

Name: **Manoj Kumar V K**
Roll Number: **CS26RESCH11009**

September 2026

1 Introduction

This report compares a standard software implementation of an AES-128-like block cipher with a hardware-accelerated version using AES-NI instructions. Both versions implement encryption and decryption using the same custom round-key generation procedure. After verifying their correctness, I benchmarked both versions to compare their performance.

The second part of this assignment involves generating the 8×8 multiplication table for the extension field $GF(2^3) = \mathbb{F}_2[x]/(x^3 + x + 1)$.

The core AES implementation includes:

1. A conventional software implementation of AES encryption and decryption.
2. An AES-NI implementation using processor-supported hardware instructions.

To get a complete picture of the performance, I ran benchmarks for all four operations across input sizes of 1 KB, 2 KB, 4 KB, 32 KB, and 64 KB.

2 System and Experimental Environment

2.1 Machine and Processor

I ran all benchmarks and tests on an ASUS ROG Zephyrus G14. The system specifications are listed in Table 1.

Table 1: Hardware and Operating System Specifications

Parameter	Specification
Manufacturer	ASUSTeK COMPUTER INC.
Model	ROG Zephyrus G14 GA401IH_GA401IH
System Type	x64-based PC
Processor	AMD64 Family 23 Model 96 Stepping 1
Processor Frequency	~2.9 GHz
Operating System	Microsoft Windows 11 Home Single Language
OS Build	10.0.26200
Total Physical Memory	15,790 MB

2.2 Compiler and Compilation Flags

The code was compiled using the MSYS2 GCC toolchain:

g++ (Rev6, Built by MSYS2 project) 16.1.0

The AES benchmark was compiled with the following command to ensure the compiler doesn't optimize away the encryption loops:

```
g++ aes.cpp -o aes -march=native -O3
```

I used the `-O3` flag for maximum optimization. Instead of manually passing `-maes` or `-msse2`, I used `-march=native` so the compiler could automatically detect and enable all the hardware instruction sets supported by my laptop's CPU.

The $GF(2^3)$ multiplication program was compiled similarly:

```
g++ multiplication_table.cpp -o multiplication_table -O3
```

3 Implementation Details

3.1 AES Software Implementation

The conventional software version implements the AES transformations (SubBytes, ShiftRows, MixColumns, AddRoundKey) directly in C++.

Instead of the standard AES key schedule, the round keys are generated recursively using the assignment's specific rule:

$$k_i = \text{ShiftRows}(k_{i-1}) \oplus RC_i$$

3.2 AES-NI Hardware Implementation

The AES-NI version skips the manual byte-shifting and Galois Field math by using compiler intrinsics that map directly to x86 hardware instructions:

- `_mm_aesenc_si128` and `_mm_aesenclast_si128`
- `_mm_aesdec_si128` and `_mm_aesdeclast_si128`
- `_mm_aesimc_si128` (used to transform the keys for decryption)

Both implementations use the exact same key schedule, which ensures they are doing identical cryptographic work and produce the same outputs.

3.3 $GF(2^3)$ Multiplication

For the second task, the program implements multiplication in $GF(2^3) = \mathbb{F}_2[x]/(x^3 + x + 1)$. Each element is represented using 3 bits corresponding to $a_2x^2 + a_1x + a_0$.

Polynomial multiplication over $\mathbb{F}_2$ is done using the logical XOR operation, and the result is reduced modulo $x^3 + x + 1$. For instance:

$$x \cdot x^2 = x^3 \equiv x + 1 \pmod{x^3 + x + 1}$$

In binary, this is represented as:

$$010 \times 100 = 011$$

The program uses this logic to dynamically print the full 8×8 multiplication table.

4 Correctness Verification

Before running the performance benchmarks, I verified the correctness of the implementations using 100 random plaintexts and keys. I tested four specific properties to ensure the software and AES-NI versions behaved correctly and identically:

$$D_{SW}(E_{SW}(P)) = P \quad (1)$$

$$D_{AESNI}(E_{AESNI}(P)) = P \quad (2)$$

$$E_{SW}(P) = E_{AESNI}(P) \quad (3)$$

$$D_{SW}(C) = D_{AESNI}(C) \quad (4)$$

All tests passed successfully, confirming that the implementations properly encrypt and decrypt the data and match each other perfectly (Table 2).

Table 2: Correctness Verification Results (100 Random Trials)

Verification Test	Result
Software $D(E(P)) = P$	PASS
AES-NI $D(E(P)) = P$	PASS
Software $E(P) = \text{AES-NI } E(P)$	PASS
Software $D(C) = \text{AES-NI } D(C)$	PASS

5 Performance Benchmark Results

I benchmarked the software and AES-NI encryption and decryption separately.

The tests were run on payload sizes of 1 KB, 2 KB, 4 KB, 32 KB, and 64 KB. To filter out operating system noise, I ran 21 independent trials for each payload size. Each trial processed the payload 1,000 times to ensure the measured time was long enough to be accurate. I reported the median execution time from these 21 runs.

Throughput was calculated as:

$$\text{Throughput} = \frac{N}{T}$$

Where N is the total bytes processed and T is the execution time in seconds. Cycles per byte (CPB) was calculated using the `_rdtsc()` instruction:

$$\text{CPB} = \frac{C_{\text{end}} - C_{\text{start}}}{N}$$

Table 3: Median Throughput (MB/s)

Implementation	1 KB	2 KB	4 KB	32 KB	64 KB
Software Encryption	88.10	88.69	88.26	87.29	86.73
AES-NI Encryption	8774.15	8841.67	8795.88	8798.36	8734.17
Software Decryption	52.85	53.55	53.37	53.16	53.03
AES-NI Decryption	8789.94	8885.92	8833.67	8771.44	8756.81

Table 4: Median Cycles Per Byte (CPB)

Implementation	1 KB	2 KB	4 KB	32 KB	64 KB
Software Encryption	31.32	31.12	31.27	31.62	31.83
AES-NI Encryption	0.31	0.31	0.31	0.31	0.32
Software Decryption	52.21	51.55	51.72	51.93	52.06
AES-NI Decryption	0.31	0.31	0.31	0.31	0.32

6 Discussion & Analysis

1. How much faster is AES-NI encryption compared with your software implementation?

AES-NI encryption is roughly 100 times faster than the conventional software implementation. For example, on the 64 KB payload, AES-NI achieved 8,734 MB/s compared to the software’s 86.7 MB/s.

2. How much faster is AES-NI decryption compared with your software implementation?

The speedup for decryption is even larger. AES-NI decryption is about 165 times faster than the software implementation. Software decryption ran at 53 MB/s, while AES-NI decryption ran at 8,756 MB/s on the 64 KB payload.

3. Is the performance improvement similar for encryption and decryption? If not, why might they differ?

No, the performance improvement is noticeably different (100x speedup for encryption vs. 165x for decryption). This happens because software decryption is inherently more computationally expensive than software encryption. Encryption uses the MixColumns step, which only requires multiplying by 2 and 3 in the Galois Field. Decryption uses Inverse MixColumns, which requires multiplying by 9, 11, 13, and 14. These larger constants require more operations in C++, which slows down the software decryption baseline. Because AES-NI handles these matrix multiplications in hardware, it doesn’t suffer this penalty, making the relative speedup for decryption look much larger.

4. Compare the throughput and cycles per byte obtained for encryption and decryption.

The throughput and CPB metrics match up perfectly (as throughput increases, CPB drops). Software encryption processes data at roughly 31 cycles per byte, while software decryption takes about 52 cycles per byte. The hardware instructions execute much more efficiently, bringing the cost down to just 0.31 cycles per byte for both AES-NI encryption and decryption.

5. How do the measured results change as the message size increases?

The performance stays mostly flat as the message size scales from 1 KB up to 64 KB. Because my benchmark loop repeats the encryption process 1,000 times per run, even the 1 KB payload is doing enough total work to saturate the CPU and minimize the overhead of reading the `_rdtsc()` timer. As a result, the throughput and CPB represent the true steady-state performance of the hardware across all tested sizes.

7 Conclusion

The benchmark results clearly show how important hardware acceleration is for cryptography. The AES-NI implementation was around 100 times faster for encryption and 165 times faster for decryption compared to the C++ implementation. By using random plaintext and key tests, I was able to confirm that these performance gains are achieved while maintaining exact cryptographic correctness.